

# **Dossier technique PETIT Côme partie individuelle**

**24/02/2026**

The logo consists of a solid dark red square. Centered within this square is the text "Benoît MARAIS" in a bold, yellow, sans-serif font. Directly beneath it, the word "MENUISERIE" is written in a smaller, bold, yellow, sans-serif font.

**Benoît MARAIS**  
**MENUISERIE**

# Sommaire

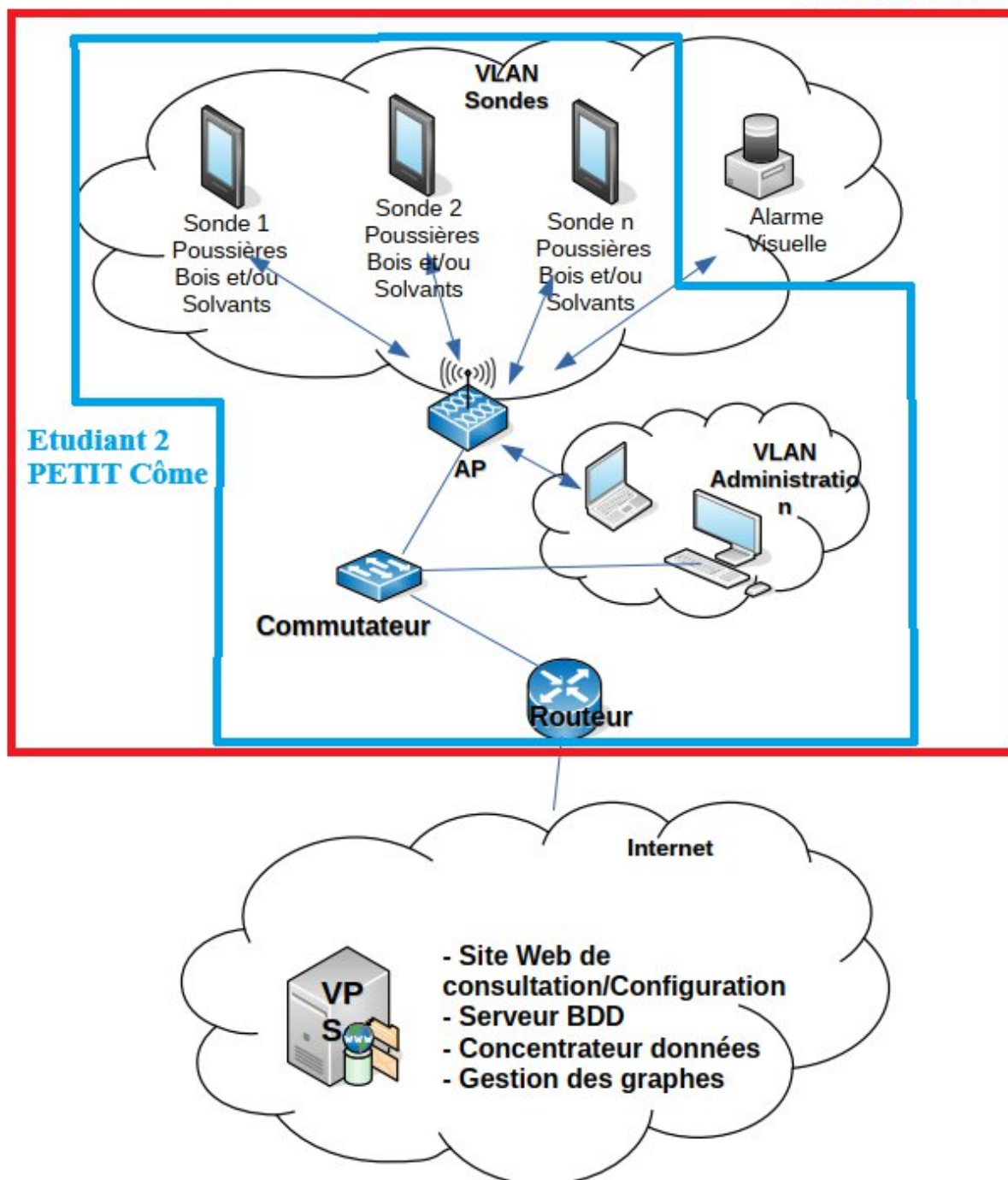
|                                                                   |    |
|-------------------------------------------------------------------|----|
| Sommaire.....                                                     | 2  |
| Situation dans le projet.....                                     | 3  |
| Synoptique de la réalisation.....                                 | 3  |
| Partie personnelle.....                                           | 7  |
| Prototypes IHM.....                                               | 9  |
| Topic broker.....                                                 | 9  |
| Découverte carte Unihiker et premier programme d'acquisition..... | 10 |
| Passage des programmes en objet.....                              | 10 |
| Test IHM sous Tkinter .....                                       | 10 |
| Réalisation des FS et des FT .....                                | 11 |
| Implémentation détaillée.....                                     | 14 |
| Test unitaire.....                                                | 16 |
| Ccs811.....                                                       | 17 |
| MQTT.....                                                         | 18 |
| MesureTVOCCO2.....                                                | 18 |
| Conclusion et problèmes rencontrés.....                           | 19 |
| Bilan de la réalisation personnelle.....                          | 20 |
| Points du projet validés.....                                     | 20 |
| Améliorations possibles.....                                      | 20 |
| Synthèse personnelle / conclusion.....                            | 20 |

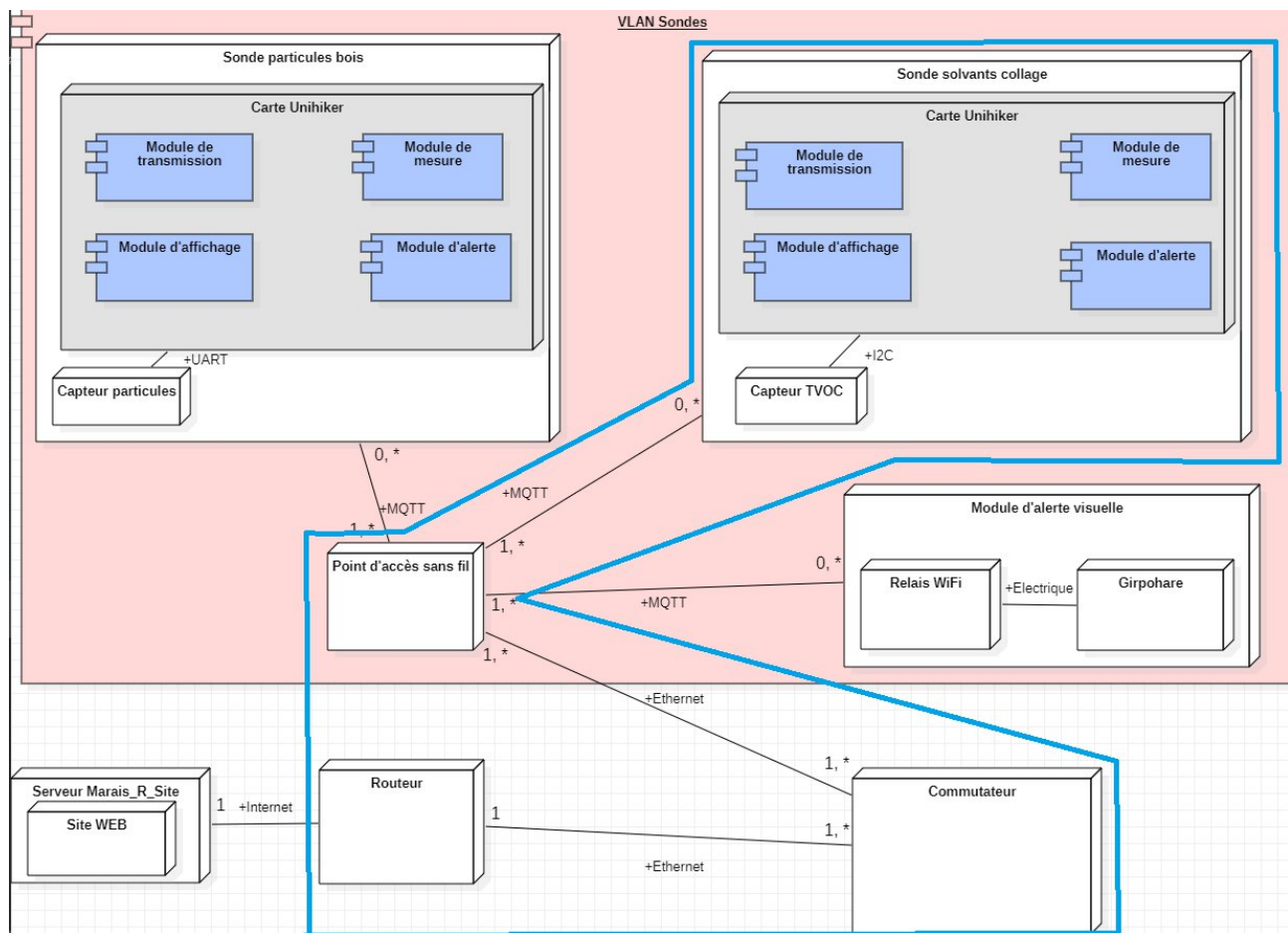
# Situation dans le projet

## Synoptique de la réalisation

La structure du projet sera une installation presque complète dans le sens où seul le router est déjà présent. Les sondes, composées d'un ou plusieurs capteurs et d'une carte Unihiker. Les alarmes visuels, point d'accès WiFi et le commutateur et ses VLAN seront installés par notre projet étant donné que dans l'infrastructure actuelle de la menuiserie aucun des différents éléments n'est présent.

Comme indiqué par le cadre bleu dans le schéma suivant, ma partie concerne les capteurs ainsi que la partie réseau (commutateur, VLAN). Plus précisément les capteurs de CO<sub>2</sub> et TVOC et la configuration du WiFi.

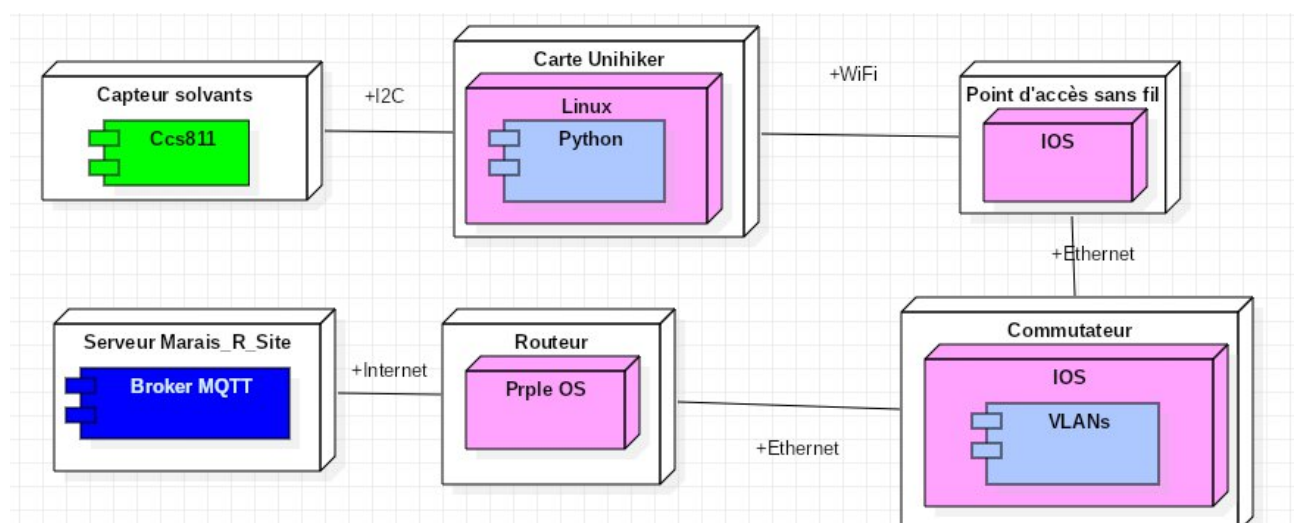




Vous pouvez constater que dans ma partie je vais me servir de plusieurs protocole de communication. I2C pour la communication entre le capteur et la carte. MQTT entre le point d'accès WIFI et la sondes. Pour finir Ethernet entre les différentes interfaces réseau.

Le but des sondes est d'envoyer au broker MQTT les mesures dans un fichier JSON.

Pour les trois interface réseau leurs rôle est de segmenter le réseau avec des VLAN. Et surtout de transmettre les mesures de manière confidentiels vers le projet MaraisRSite.



Vous pouvez voir dans le schéma ci dessus un total de six nœuds. Le premier est celui du capteur et de la carte Unihiker. Les deux éléments communiquent en I2C avec la carte en maître et le capteur en esclave. La carte se connecte en wifi au point d'accès qui lui fournit une configuration IP à l'aide du DHCP. Le commutateur fait le lien entre le réseau local et la box nous permettant d'aller sur internet. Le VPS de marais'rsite met à disposition le broker pour pouvoir publier les mesures des capteurs.

# Partie personnelle

Objectifs :

- Prise en main et configuration des sondes (TVOC et CO2)
- Codage des IHM conforme au cahier des charges et au différentes fonction technique et au fonction de service
- Réalisation des tests unitaires des différents codes présents
- Réalisation des tests d'intégration

|                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Étudiant 2</p> <p>ER <input type="checkbox"/></p> <p>IR <input checked="" type="checkbox"/></p> | <p>La réalisation des fonctions de services, techniques et des scénarios associées : FS2 FS4 FS6 FS7 FT2 FT3,2 FT3,3 FT5,1 FT7</p> <p>La réalisation des recettes techniques et fonctionnelles associées et au moins un test unitaire.</p> <p>L'étude et le respect des contraintes associées</p> <p>- Dev Python/Kivy :Capteur TVOC / IHM</p> <p>- <i>Wifi</i></p> <p>- <i>MQTT</i></p> | <p><b>Installation :</b></p> <p>Environnement de développement carte Unihiker (R1)</p> <p>Firmware, Bibliothèques (R1)</p> <p><b>Configuration et mise en œuvre :</b></p> <p>AP (R1), Capteur TVOC(R2)</p> <p><b>Analyse :</b> Diagramme Sysml/UML préliminaires (R1),</p> <p><b>Conception :</b> Analyse détaillées UML (R1), Plan Tests unitaires/fonctionnels (R2), Plan d'intégration (R2)</p> <p><b>Réalisation :</b> Codage des FS et FT (R3), tests unitaires(R3), Test d'intégration (R3)</p> <p><b>Documentation :</b> Dossier de spécification (R1/R2), Dossier de réalisation/recette (R2/R3), Manuels (R2/R3)</p> |
|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Avenant du 26/03/2026

Nouvelle répartition des FS et FT :

FS5 (Etudiant 3) Visualiser facilement les dépassements de seuils sur la carte Unihiker  
→ Etudiant 1/2

FS7 (Etudiant 1/2/3) Mettre en œuvre une architecture réseau sécurisée et segmentée →  
Etudiant 3

FT3.4 (Etudiant 3): Programmer les Unihiker pour récupérer et afficher les seuils →  
Etudiant 1/2

FT5.3 (Etudiant 3): Définir le message MQTT pour la réception des seuils →  
Etudiant 1/2

FT6 (Etudiant 1/3): Configurer un réseau segmenté avec VLANs → Etudiant 3

FT7 (Etudiant 2): Mettre en place des points d'accès WiFi → Etudiant 3

FT8 (Etudiant 1): Installer un commutateur de niveau 2 pour gérer les VLANs →  
Etudiant 3

FT10 (Etudiant 3) : Intégrer un module de visualisation des dépassements de seuils sur  
la carte Unihiker → Etudiant 1/2

L'étudiant3 est chargé spécifiquement de réaliser les scripts de tests d'infrastructure :

- Test infra ou solution équivalente:
  - pour vérifier la configuration des équipements de l'architecture réseau (vlan, ACL, routage, DHCP ...) avec NAPALM ou Netmiko par exemple
  - pour vérifier la disponibilité des services réseaux nécessaires au projet (broker MQTT)
  - pour vérifier la disponibilité des équipements listés dans la BDD (sondes, alarme) avec arp-scan par exemple

## Appropriation du cahier des charges

L'appropriation du cahier des charges est passé par le création de plusieurs digrammes notamment UML. Le but étant de mettre en exergue les acteurs et les cas d'utilisation par exemple. Nous donc crée le digramme de cas d'utilisation pour nous répartir les tâches et identifier les différents acteurs, le diagramme d'exigence pour faire apparaître les contrares et le diagramme de déploiement pour faire apparaître les équipement et leurs logiciels.

## Prototypes IHM

Le but du prototype d'IHM est de comprendre les besoin et les attentes du client et ses besoins, sans pour autant coder IHM. Lors de notre visite dans la menuiserie notre client a valider la forme de nos IHM. Il a également donné son avis sur l'affiche de plusieurs valeurs a la fois, il préférerait que les différentes valeurs soit afficher une a la fois et alternant tous les x temps. Voici un exemple de celui-ci :



## Topic broker

| Topic              | Description du topic                                     |
|--------------------|----------------------------------------------------------|
| marais/sondes/MAC  | Contient la mesure d'une carte indiquée son adresse MAC. |
| marais/alarmes/MAC |                                                          |
| marais/seuils/TVOC |                                                          |
| marais/seuils/CO2  |                                                          |
| marais/seuils/PM10 |                                                          |

## **Exemple JSON :**

Mesure

```
payload = { "timestamp": ts, "mesure" : {"ECO2":eco2},{ "TVOC":tvoc} }
```

Seuils

```
payload = { "timestamp": ts, "seuil" : {"TVOC":40} }
```

## **Découverte carte Unihiker et premier programme d'acquisition**

Découverte de la carte Unihiker. Elle utilise son propre OS modifié qui a pour base un debian 10. Utilisation des programme de démo pour tester les fonctionnalisées. Des outils pour coder notamment Jupyter le ssh et le smb pour le partage de fichiers. Découverte de la bibliothèque native de la carte Unihiker PingPong. Utile notamment pour les boutons. Pour acquérir il est nécessaire d'utiliser la bibliothèque pingpong et I2C pour communiqué avec le capteur. Les premiers programme était en procédural.

## **Passage des programmes en objet**

Le passage des programmes en objet a permit de s'imprégner des différentes fonction qui sont devenue des méthode. Ce passage a permit de crée des programmes qui plus tard feront parti du dossier model d'un système model view controller. J'ai donc séparer le programme en trois classe la première Ccs811 qui gère le registre du capteur et son initialisation, la deuxième MesureTVOC\_CO2 qui gère la période de chauffe du capteur appeler run-in, le déclenchement des mesures et l'acquisition. La troisièmes est MQTTClient elle gère la connexion au broker et la publication des mesures.

## **Test IHM sous Tkinter**

J'ai testé de coder l'IHM sous Tkinter, au final nous retiendrons l'IHM sous Kivy même si il est plus demandeur en ressource il est plus ergonomique et complet que Tkinter.

# Réalisation des FS et des FT

Fonctions services :

FS2 : Mesurer en temps réel les niveaux de solvants (CO<sub>2</sub>, TVOC) dans les zones de collage et peinture.

FS4 : Contribuer à l'affichage clair et intuitif des données sur les Unihiker (identification, valeurs, seuils).

FS6 : Participer aux mesures de sécurisation et chiffrement des échanges (MQTT/TLS, authentification).

Fonctions techniques

FT2 : Installer des capteurs CO<sub>2</sub> et TVOC dans les zones de collage et peinture.

FT3.2 : Programmer les Unihiker pour afficher en temps réel les valeurs CO<sub>2</sub> et TVOC (IHM).

FT3.3 : Participer à la configuration de l'affichage de l'identification de la sonde sur l'IHM (adresse MAC).

FT5.1 : Définir le message MQTT pour l'envoi des valeurs de mesure (structure, champs, sécurité).

## **FS2 : Mesurer en temps réel les niveaux de solvants (CO2, TVOC) dans les zones de collage et peinture.**

Ce cas d'utilisation décrit comment à l'aide du capteur la carte Unihiker est capable de mesurer le niveau de solvant.

Le choix s'est porté sur la carte Unihiker et le capteur Ccs811v1. Le capteur permettant à la fois de mesurer le taux de TVOC et de CO2 pour le même prix que les autres capteurs du marché mais avec plus de polyvalence car il fait les deux à la fois.

Les bibliothèques utilisées sont la bibliothèque du capteur et la bibliothèque I2C.

## **FS4 : Contribuer à l'affichage clair et intuitif des données sur les Unihiker (identification, valeurs, seuils).**

Ce cas d'utilisation permet un affichage clair via l'IHM sur la carte Unihiker. Le choix a été la carte Unihiker pour un affichage LCD en plus de sa compatibilité I2C avec le capteur.

Pour l'interface graphique la bibliothèque utilisée est Kivy. D'autres bibliothèques tels que Tkinter ou CustomTkinter ont été testées mais offraient moins de fonctionnalités même si Kivy en échange de ses fonctionnalités consomme plus de mémoire RAM et de charge du processeur. Même si bien gérer on peut limiter cette charge et donc profiter pleinement des avantages de Kivy sans ses inconvénients.

## **FS6 : Participer aux mesures de sécurisation et chiffrement des échanges (MQTT/TLS, authentification).**

Ce cas d'utilisation demande un chiffrement des échanges afin que les valeurs ne soient pas en clair.

Le choix a été d'utiliser le MQTTS protocole éponyme du MQTT mais qui rajoute une couche de chiffrement TLS à l'aide d'un certificat. Il est également nécessaire d'avoir un login et un mot de passe pour se connecter au broker.

## **FT2 : Installer des capteurs CO2 et TVOC dans les zones de collage et peinture.**

Installation des capteurs dans les zones de collage et peinture de la menuiserie.

## **FT3.2 : Programmer les Unihiker pour afficher en temps réel les valeurs CO2 et TVOC (IHM).**

Cette fonction service a été réalisé en deux phases. La première était de coder en procédural pour faire des test notamment pour récupérer les mesures et les envoyer en MQTT au broker. Puis les programme ont été mis au propre en objet. Chaque classe fait une tache une classe pour les mesures, le MQTT et l'IHM. Le tous dans un modèle : « Model, View, Controller »

## **FT3.3 : Participer à la configuration de l'affichage de l'identification de la sonde sur l'IHM (adresse MAC).**

Cette fonction a été coder et est accessible via un bouton permettant a l'accès a une page pour les technicien avec l'adresse MAC l'adresse IP et d'autre information sur le réseau notamment la qualité du signal permettant de savoir si la carte peut être placé a certain endroit.

## **FT5.1 : Définir le message MQTT pour l'envoi des valeurs de mesure (structure, champs, sécurité).**

Cette fonction était primordiale pour l'échange de donnée entre MaraisRsense et MaraisRsite. Le but étant de publier dans le brocker des topic clair en permettant l'extraction des ces données par MaraisRsite. Le topic comprenant l'adresse MAC en tant que nom de sondes pour facilitée identification car par définition l'adresse MAC est unique. Le tous prenant en compte que une sondes peut avoir plusieurs types de compteur, il était donc nécessaire de garder une structure commune quelque soit la valeurs mesuré par le capteur. TVOC/CO2 ou PM10 ou les deux a la fois.

## Implémentation détaillée

La fonction FT3.2 consiste à programmer la carte UniHiker afin d'afficher en temps réel les valeurs de CO<sub>2</sub> et de TVOC mesurées dans les zones de collage et de peinture. Cette fonction participe directement à la fonction de service FS2, qui vise à mesurer en temps réel les niveaux de solvants, ainsi qu'à la fonction FS4 liée à l'affichage clair et intuitif des données sur l'IHM.

Le choix matériel s'est porté sur la carte UniHiker associée au capteur CCS811. La carte UniHiker a été retenue car elle intègre un écran permettant un affichage clair et intuitif comme demandé, tout en restant compatible avec une communication capteur en I2C et avec un développement Python. Le capteur CCS811 a été choisi car il permet de mesurer à la fois le TVOC et l'eCO<sub>2</sub> avec un seul composant, ce qui simplifie l'intégration matérielle et logicielle, rationalise également les coup.

L'architecture logicielle a été organisée en MVC afin de séparer les tâches. La classe Ccs811 gère l'initialisation du capteur et l'accès à ses registres via le bus I2C. La classe MesureTVOCCO2 pilote la phase de chauffe du capteur, déclenche les acquisitions périodiques et formate les valeurs mesurées. Enfin, la classe MQTTClient gère la connexion au broker, l'authentification et la publication sécurisée des mesures (TLS 1.2).

Les données traitées par la sonde sont les valeurs d'eCO<sub>2</sub>, de TVOC, l'horodatage de la mesure ainsi que l'identifiant unique de la sonde, représenté par son adresse MAC. Lors de l'envoi vers le broker MQTT, les mesures sont encapsulées dans une structure JSON afin de faciliter leur exploitation côté serveur. Cette structure commune permet de conserver un format homogène, quelle que soit la nature du capteur utilisé dans la sonde.

L'interface homme-machine a été développée avec Kivy. Ce choix a été préféré à Tkinter après expérimentation, car Kivy offre une interface plus moderne et plus souple pour afficher les valeurs mesurées. En contrepartie, Kivy consomme davantage de ressources, ce qui a nécessité une attention particulière sur la fréquence de rafraîchissement et l'organisation du code.

La méthode principale d'acquisition repose sur une lecture cyclique. Après la phase de chauffe initiale(run-in), le programme lit les mesures eCO<sub>2</sub> et TVOC, contrôle leur validité, met à jour l'affichage local puis prépare une charge utile JSON envoyée au

broker MQTT. Cette publication permet ensuite au projet MaraisRSite de stocker et d'exploiter les données à distance.

Voici un exemple de code permettant de voir toute les fonctionnalisées précédemment données :

```
def publish_measure(self):
    eco2, tvoc = self.sensor.read_data()
    timestamp = datetime.now().isoformat()

    payload = {
        "ts": timestamp,
        "measure": {
            "ECO2": eco2,
            "TVOC": tvoc
        }
    }

    topic = f"marais/sondes/{self.mac_address}"
    self.mqtt_client.publish(topic, json.dumps(payload))
    self.view.update_values(eco2, tvoc)
```

**read\_data()** : lit les valeurs du capteur.

**timestamp** : permet de dater la mesure.

**payload** : structure les données au format JSON.

**topic** : utilise l'adresse MAC pour identifier de façon unique la sonde.

**publish()** : envoie la mesure au broker MQTT.

**update\_values()** : rafraîchit l'affichage local.

# Test unitaire

Afin de valider les modules développés dans ma partie personnelle, j'ai mis en place plusieurs tests unitaires sur les classes principales du projet liées à l'acquisition des mesures CO2/TVOC et à leur transmission. Ces tests ont été réalisés en Python avec le framework unittest et l'utilisation de unittest.mock, ce qui a permis de simuler le matériel et les échanges sans dépendre en permanence du capteur réel ou du broker MQTT. Ne pas dépendre d'autres système ou éléments physique est primordiale car si on dépend d'autres choses le test perd de son sens car il ne test plus uniquement ce qu'on voulait tester.

Les tests unitaires ont été organisés de manière chronologique autour de trois classes principales. La première série concerne la classe Ccs811, chargée de la communication bas niveau avec le capteur via I2C. La deuxième série concerne la classe MQTT, responsable de la connexion, de la publication et de l'abonnement aux topics. Enfin, la troisième série concerne la classe MesureTVOCCO2, qui regroupe la logique d'initialisation matérielle, de période de chauffe du capteur et de récupération des mesures.

Les objectifs de ces tests étaient de vérifier le fonctionnement nominal des méthodes, de valider la gestion de certaines erreurs, et de s'assurer que les valeurs renvoyées ou les appels réalisés correspondaient au comportement attendu. Les résultats d'exécution montrent que l'ensemble des tests prévus ont été validé.

| Référence | Élément testé        | Date       | Objectif                                                                                               |
|-----------|----------------------|------------|--------------------------------------------------------------------------------------------------------|
| TU-CCS811 | Classe Ccs811        | 30/03/2026 | Vérifier l'initialisation, la lecture/écriture I2C, le décodage eCO2/TVOC (scénarios simulés).         |
| TU-MQTT   | Classe MQTT          | 31/03/2026 | Vérifier l'initialisation client MQTT, la publication et l'abonnement (mock).                          |
| TU-MESURE | Classe MesureTVOCCO2 | 01/04/2026 | Valider l'initialisation hardware, la logique run-in, la lecture des mesures, gestion des erreurs I2C. |

| Outil / Environnement                                         | Résultat obtenu                                   |
|---------------------------------------------------------------|---------------------------------------------------|
| Python 3.x, unittest<br>+<br>unittest.mock (simulation I2C)   | Conforme — 8 tests exécutés et validés.           |
| Python 3.7, unittest + mock du client<br>paho.mqtt.client     | Conforme — 3 tests exécutés et validés.           |
| Python 3.x, unittest +<br>unittest.mock (simule Board/Ccs811) | Conforme — 6 tests exécutés et validés (rapport). |

## Ccs811

Le test unitaire de la classe Ccs811 a été réalisé le 30/03/2026 avec le framework unittest. Son objectif était de vérifier le fonctionnement nominal des différentes méthodes de la classe utilisées pour la détection de CO2 et de TVOC, notamment l'initialisation du capteur, la lecture et l'écriture dans les registres I2C, ainsi que la lecture simulée des mesures.

L'environnement de test était constitué des fichiers Ccs811.py et testccs811.py, d'un environnement Python 3.14.5, et d'une simulation du bus I2C au moyen de mocks. Cette approche a permis de tester le comportement logiciel sans nécessiter le capteur physique pendant toutes les phases de validation.

La procédure de test consistait à exécuter la commande `python -m unittest testccs811 -v`, puis à contrôler les résultats affichés dans la console. Plusieurs cas ont été vérifiés, comme la lecture d'un registre simulé, l'écriture d'un registre, la détection de disponibilité des données, le décodage des valeurs eCO2/TVOC et l'initialisation correcte du capteur.

Le rapport d'exécution montre que 8 tests ont été exécutés en 0,012 seconde et que tous ont été validés. Cette campagne m'a permis de confirmer que les méthodes principales de la classe Ccs811 fonctionnaient correctement dans les scénarios prévus.

## **MQTT**

Le test unitaire de la classe MQTT a été réalisé le 31/03/2026. Il avait pour objectif de vérifier que la création de l'objet initialise correctement le client MQTT, que la méthode de publication transmet bien un message, et que la méthode d'abonnement enregistre correctement une souscription à un topic.

Le test a été mené en Python 3.7 avec unittest et un client MQTT simulé. La procédure consistait à lancer le programme de test dédié, puis à observer les appels réalisés vers le client simulé lors de l'initialisation, de la publication et de l'abonnement.

Les résultats obtenus sont conformes aux résultats attendus pour les trois cas de test TU1.1, TU1.2 et TU1.3. Cela valide le bon fonctionnement de la classe pour la communication avec le broker dans les cas nominaux.

## **MesureTVOCCO2**

Le test unitaire de la classe MesureTVOCCO2 a été réalisé le 01/04/2026. Son objectif était de valider l'initialisation matérielle, la logique de récupération des mesures, la gestion de la période de chauffe du capteur, ainsi que le comportement du programme en cas d'erreur I2C ou d'initialisation échouée.

L'environnement de test reposait sur Python 3.x, unittest.mock et une simulation de la carte et du capteur BoardCcs811I2C, sans matériel réel. La procédure consistait à exécuter `python -m unittest testMesureTVOCCO2.py -v` puis à contrôler la conformité des résultats affichés.

Les scénarios testés sont particulièrement intéressants car ils ne se limitent pas au fonctionnement nominal. Ils vérifient aussi le comportement lorsque ccs811 vaut None, lorsqu'une erreur I2C survient, lorsque la période de run-in n'est pas terminée, ou lorsque les données sont enfin prêtes et lisibles.

Le rapport d'exécution mentionne 6 tests exécutés en 4,159 secondes, tous validés. Cette campagne montre que la logique métier de la classe est robuste, notamment pour la gestion des cas particuliers liés au matériel.

## **Conclusion et problèmes rencontrés**

Lors de la mise en place des tests unitaires, la principale difficulté a été la dépendance au matériel réel, en particulier au capteur CCS811 et au bus I2C. Pour contourner ce problème, j'ai utilisé des objets simulés avec `unittest.mock`, ce qui m'a permis de tester les méthodes de façon reproductible. J'ai également rencontré la contrainte liée à la période de chauffe du capteur, qui a nécessité la création de scénarios spécifiques afin de vérifier que le programme ne retourne pas de mesures exploitables avant la fin du délai de run-in.

Ces difficultés rencontrées lors des tests sera par la suite régler car en accord avec le gérant de l'entreprise. Car comme pour l'utilisation d'un thermomètre par exemple on est bien conscient que la valeurs de la température ne vas pas changer instantanément en cas de changement d'environnement. La période de run-in sera donc réduite de 2 minute a trente seconde pour quand même laisser la temps a la cellule photovoltaïque de chauffer tout en permettant des mesure dans des temps raisonnable dans le cas du démarrage du programme.

# Bilan de la réalisation personnelle

## Points du projet validés

Intégration matérielle et prise en main de la carte UniHiker et du capteur CCS811 : la carte a été découverte, les échanges I2C testés et l'identification du capteur (HWID) validée.

Développement de la couche bas-niveau capteur (classe Ccs811) : initialisation, lecture/écriture de registres, test de disponibilité des données et décodage des valeurs eCO2/TVOC ont été implémentés et validés par tests unitaires.

Implémentation de la logique de mesure (classe MesureTVOCCO2) : gestion de la phase de run-in, périodicité des acquisitions, gestion des erreurs I2C et ré-initialisation du capteur ont été codées et testées.

Communication MQTT sécurisée : formatage des messages JSON, construction des topics basés sur l'adresse MAC et implémentation d'un client MQTT ont été réalisés.

Tests unitaires et stratégie de validation : plans de tests, fiches de tests et exécution des campagnes unitaires pour Ccs811, MQTT et MesureTVOCCO2.

## Améliorations possibles

Sécurité : automatiser le déploiement et la rotation des certificats TLS, ajouter la validation mutuelle (mTLS) si le broker et l'architecture le permettent.

Optimisation énergie / performances : réduire la fréquence d'émission ou mettre en place un mode basse consommation si requis par contraintes matérielles.

Scalabilité : définir et tester une stratégie de sharding/partition des topics ou d'équilibrage du broker si le déploiement dépasse plusieurs dizaines/centaines de sondes.

## Synthèse personnelle / conclusion

Le projet était clair et m'a apporté beaucoup quand à la gestion d'un travail de groupe et de gérer les besoins d'un client. Le projet m'a permis de progresser en programmation et de gérer un code robuste, clair, et organiser en **Modèle Vue Contrôleur** afin de coder de manière professionnelle. Un grand merci à M AVRIL d'avoir été clair dans ses demandes à l'écoute de nos interrogations vis à vis du projet.